Java — Orientação a Objetos

Parte 2 — Classes, Objetos, Encapsulamento e Herança

O coração do Java — como modelar o mundo real em código

#JAVA #OOP #CLASSES #HERANÇA

Made with GAMMA

Modelando o mundo real em código

Mundo real → Código Java

- 🐕 Cachorro → `classe Cachorro`
- 🐕 Rex (meu cachorro) → `objeto new Cachorro()`
- Nome, raça, idade → **atributos**
- Latir, correr → **métodos**

4 Pilares da OOP

📦 Encapsulamento

Esconder detalhes internos

🧬 Herança

Reaproveitar código de outra classe

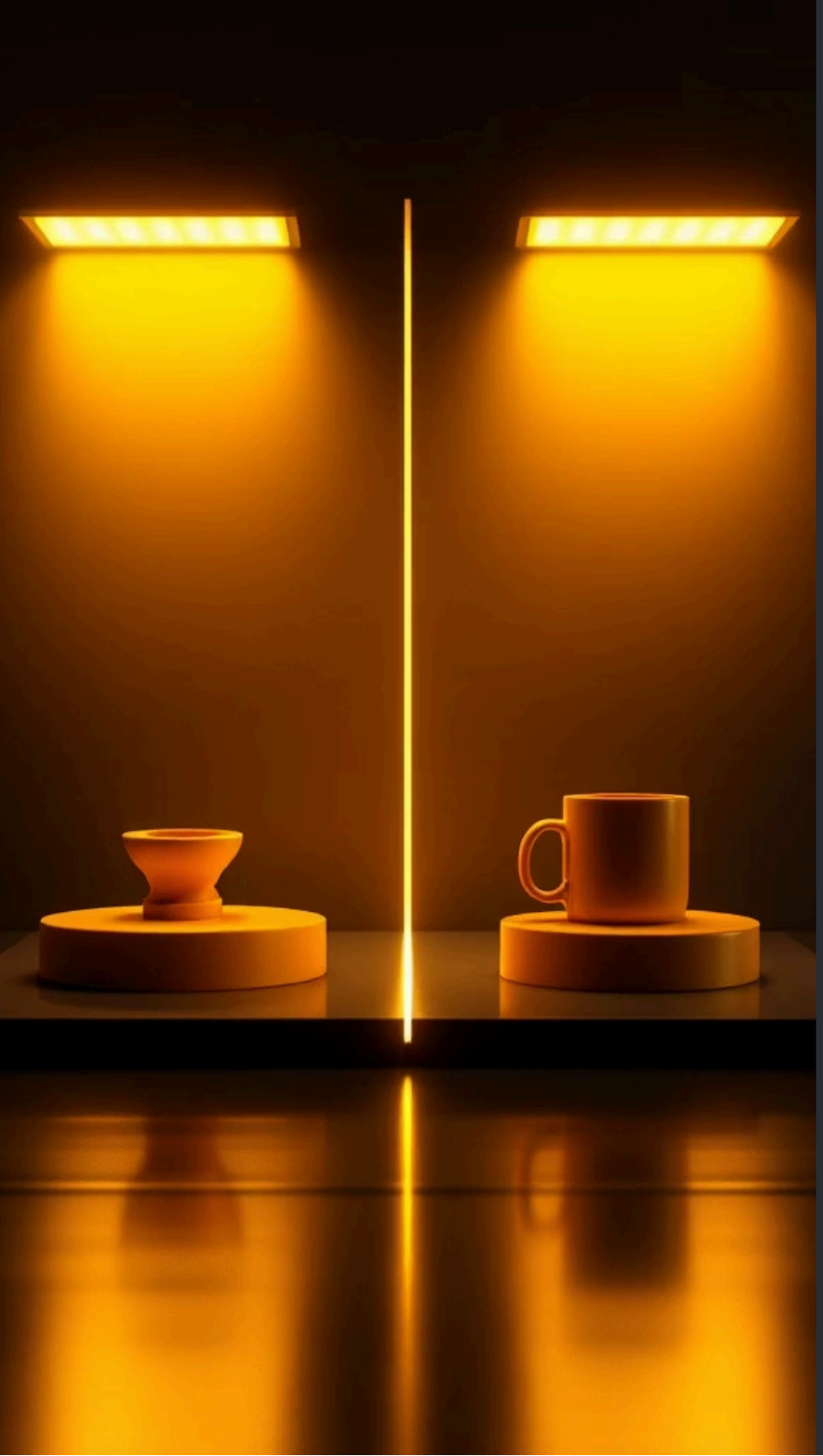
🎭 Polimorfismo

Mesmo método, comportamentos diferentes

⬜ Abstração

Mostrar só o que importa

Made with GAMMA



Classe = molde | Objeto = produto

```
// Classe — o molde
public class Pessoa {
   // Atributos
   String nome;
   int idade;

   // Método
   public void apresentar() {
      System.out.println("Olá, sou " + nome + " e tenho " + idade + "
anos.");
   }
}
```

```
// Criando objetos a partir da classe
Pessoa p1 = new Pessoa();
p1.nome = "Ana";
p1.idade = 25;
p1.apresentar(); // Olá, sou Ana e tenho 25 anos.
```

Made with GAMMA

Inicializando objetos com construtor

```
public class Pessoa {
    String nome;
    int idade;

    // Construtor — executado ao criar o objeto
    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    public void apresentar() {
        System.out.println("Olá, sou " + nome);
    }
}
```

```
// Criando objeto com construtor
Pessoa p1 = new Pessoa("Ana", 25);
p1.apresentar(); // Olá, sou Ana
```

💡 **Dica: this.** referencia o atributo da própria classe, diferenciando do parâmetro.

Made with GAMMA

Protegendo os dados da classe

```
public class ContaBancaria {
    private double saldo; // privado — só acessível dentro da classe

    // Getter — lê o valor
    public double getSaldo() {
        return saldo;
    }

    // Setter — altera o valor com validação
    public void depositar(double valor) {
        if (valor > 0) {
            saldo += valor;
        }
    }
}
```

```
ContaBancaria conta = new ContaBancaria();
conta.depositar(100.0);
System.out.println(conta.getSaldo()); // 100.0
```

`private`

Ninguém de fora acessa diretamente

`get/set`

Porta controlada para ler e alterar

Made with GAMMA

Reaproveitando código com herança

```
// Classe pai (superclasse)
public class Animal {
    String nome;

    public void respirar() {
        System.out.println(nome + " está respirando.");
    }
}

// Classe filha (subclasse) — herda de Animal
public class Cachorro extends Animal {
    public void latir() {
        System.out.println(nome + " está latindo.");
    }
}
```

```
// Uso
Cachorro rex = new Cachorro();
rex.nome = "Rex";
rex.respirar(); // herdado de Animal
rex.latir();    // próprio de Cachorro
```

💡 **Dica:** Use **extends** para herdar. A subclasse recebe tudo da superclasse.

Made with GAMMA

Mesmo método, comportamentos diferentes

```
public class Animal {
    public void emitirSom() {
        System.out.println("Som genérico");
    }
}

public class Cachorro extends Animal {
    @Override
    public void emitirSom() {
        System.out.println("Au au!");
    }
}

public class Gato extends Animal {
    @Override
    public void emitirSom() {
        System.out.println("Miau!");
    }
}
```

```
// Polimorfismo em ação
Animal a1 = new Cachorro();
Animal a2 = new Gato();
a1.emitirSom(); // Au au!
a2.emitirSom(); // Miau!
```

💡 **Dica: @Override** indica que o método está sendo sobrescrito da classe pai.

Made with GAMMA

Classes Abstratas e Interfaces

Classe Abstrata

```
// Não pode ser instanciada diretamente
public abstract class Forma {
    public abstract double calcularArea();
}

public class Circulo extends Forma {
    double raio;
    @Override
    public double calcularArea() {
        return Math.PI * raio * raio;
    }
}
```

Interface

```
// Contrato que a classe deve cumprir
public interface Voavel {
    void voar();
}

public class Passaro implements Voavel {
    @Override
    public void voar() {
        System.out.println("Voando!");
    }
}
```

abstract

Modelo incompleto que obriga a implementação

interface

Contrato de comportamento

Made with GAMMA

Revisando os 4 pilares da OOP

🔒 Encapsulamento

Atributos private + métodos get/set

Protege os dados internos da classe

🧬 Herança

extends → filho herda do pai

Reaproveita código sem repetição

🎭 Polimorfismo

@Override → mesmo método, resultado diferente

Flexibilidade no comportamento

⬜ Abstração

abstract / interface → define o contrato

Esconde complexidade, expõe só o essencial

Made with GAMMA

Orientação a Objetos em Java

Resumo da Parte 2

✅ **Classe = molde** | Objeto = instância

✅ **Construtor** inicializa o objeto

✅ `this.` referencia atributos da classe

✅ `private` + `get/set` = encapsulamento

✅ `extends` = herança

✅ `@Override` = polimorfismo

✅ `abstract` = classe incompleta

✅ `interface` = contrato de comportamento

➡️ **Próximo:** Arrays, ArrayList, Exceções e Boas Práticas

Made with GAMMA